



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich



A Mathematical Formulation of Cost-Aware Buffer Placement for Dataflow Circuit Optimization

Semester Project

Yuan Qin
yuaqin@student.ethz.ch

Digital Systems and Design Automation (DYNAMO)
Computer Engineering and Networks Laboratory
ETH Zürich

Supervisors:
Jiahui Xu
Prof. Dr. Lana Josipović

August 18, 2024

Abstract

Dataflow circuits offer significant advantages in managing variable latencies, unpredictable memory dependencies, and irregular control flows. The placement of buffers substantially impacts the timing and throughput of dataflow circuits, thus becoming a focal point of research in this area. However, the primary bottleneck for dataflow circuits arises from their area cost, with buffer sizes occupying a substantial proportion of the entire circuit. Therefore, reducing the area cost of buffers while maintaining high throughput is highly valuable and becomes a key consideration in optimizing dataflow circuits. This work adopts a broader range of buffer types with the potential to reduce area cost. We model their performance and integrate this model into the optimization model of dataflow circuits, while also incorporating buffer area cost into the objective. This approach enables the optimization model to achieve the lowest possible buffer area cost while ensuring maximum throughput. Experiments show that our model achieves a reduction of area overhead by 4.8-25.5% compared with the previous work under a certain clock period constraint.

Additionally, this work proposes an optimization flow for commonly found variable latency/initiation interval (II) units in dataflow circuits, ensuring optimal throughput under all variations of unit latency/II, and minimizing area cost. Furthermore, by analyzing the effects of varying latency/II on the optimization model, a systematic method is designed to reduce the computational cost of solving the optimization model.

Contents

Abstract	i
1 Introduction	1
2 Background	2
3 A New Model for Buffers	4
3.1 Reevaluating the Properties of Various Types of Buffers	4
3.2 Minimum and Maximum Latency Analysis	4
3.3 MILP Model for Performance and Area Optimization	7
4 Optimization Model for Variable Units	12
4.1 Basis of the Analysis	14
4.2 Variable Initiation Interval	17
4.3 Variable Latency	20
4.4 Divide a CFDFC to Regions	23
4.5 A Comprehensive MILP for Variable Units	25
5 Evaluation	28
5.1 Methodology	28
5.2 Baseline Descriptions	28
5.3 Comparison	29
5.4 Runtime	37
6 Conclusion	38
Bibliography	39

1 Introduction

Dynamic scheduling in high-level synthesis (HLS) tools enables dataflow circuits to adapt to runtime conditions, such as variable latencies and data dependencies, which can significantly enhance computational throughput and resource efficiency. This adaptability relies critically on the effective implementation of buffers, which serve to manage data flow and synchronization between computational elements within the circuit. Previous buffer designs in dataflow circuits [1] typically include both nontransparent and transparent buffers, offering substantial elasticity to accommodate a range of operational scenarios. However, these features often come at the cost of increased area overhead.

We observed that the current buffer placement strategies are not necessary to achieve the desired throughput. We present a new buffer performance model that supports a variety of buffer types. Our modeling strategy accurately models those less elastic buffers; this enables us to devise a buffering strategy that produces more area-efficient circuits without a performance penalty. Our work builds on these findings by exploring the potential of less elastic, more constrained buffer types that reduce area costs without compromising the throughput of the entire circuit. During this process, we discovered that reduced buffer elasticity poses a greater challenge for units with variable latency and II in meeting the goal of maximizing throughput. Our work have carefully considered and addressed this issue. The following are our contributions:

- Expanded the existing optimization model to include a broader spectrum of buffer types.
- Reconstructed throughput constraints on buffers by analyzing the relationship between latency ranges of various buffer types and circuit throughput.
- Incorporated the total area overhead of buffers into the optimization objectives with the goal of minimizing this overhead.
- Rigorously analyzed which scenarios must be considered in the optimization process and which can be disregarded, considering units with variable latency and II.
- Further expanded the cost-aware model into a two-step optimization flow to ensure that buffer placement consistently maintains maximum possible throughput in the presence of variable latency and II units.

This work examines the feasibility of using a broader range of buffer designs in dataflow circuits, aiming to maintain performance while reducing hardware costs. The scalable optimization framework is compatible with future buffer designs, aiding practical advances in HLS tools.

2 Background

In this chapter, we introduce the basic concepts related to the dataflow circuits we are working on. These include the concept of the Choice-free Dataflow Circuit (CFDFC), as well as the dataflow units and buffers within it.

CFDFC

A CFDFC is a specialized type of dataflow architecture designed to operate without conditional branching within its processing flow. In this architecture, tokens represent data units that are processed and moved through the system. Unlike common dataflow architectures that may incorporate conditional branches and control merges, CFDFC streamline processing by ensuring a predetermined path for these tokens through the network of nodes and connections. This architecture is particularly advantageous for applications requiring high throughput.

Dataflow Units in CFDFC

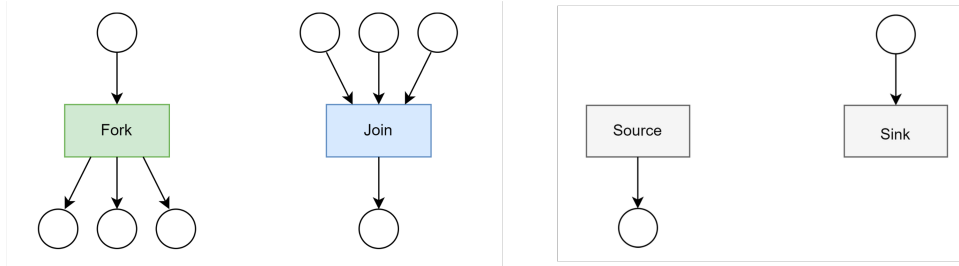


Figure 1: CFDFC Units

Dataflow circuits utilize specialized control units to manage the flow of data between functional units. Figure 1 illustrates the types of dataflow units typically found in a CFDFC due to its predetermined nature. They're explained below.

- The eager fork replicates an incoming token, distributing it immediately to all ready successors. It tracks to which successors the token has been sent and accepts a new token once all have received it.
- The lazy fork functions similarly to the eager fork but waits to distribute tokens until all successors signal readiness.
- The join unit releases a token to its successor only when tokens are available at all its inputs.
- The source unit generates an unlimited number of tokens, while the sink unit can consume an unlimited number.

Buffers

Buffers within dataflow circuits play a critical role by holding tokens until they are needed by downstream units. Specifically, token occupancy, which refers to the average number of tokens that a buffer holds over a given period, is a crucial factor. This directly impacts the circuit's throughput and latency. Thus, proper management of token occupancy is vital for optimizing the performance and efficiency of dataflow circuits.

3 A New Model for Buffers

In this chapter, we discuss our buffer performance model. We introduce two key concepts—*minimum latency* and *maximum latency*—which indicate the buffer’s behavior in transmitting tokens. These metrics also relate the channel’s token occupancy to the II of a CFDFC. We first characterize each buffer type using these metrics and then propose an optimization model that includes the constraints induced by the aforementioned characteristics of buffers. This model aims to maximize the throughput and minimize the area cost of the buffers.

3.1 Reevaluating the Properties of Various Types of Buffers

To introduce the two key metrics in our analysis, we define minimum latency L_{min} as determined by the intrinsic properties of the unit, and maximum latency L_{max} as influenced by the circuit’s II. These metrics help us devise performance models for various buffer types.

We categorize existing buffer types into four distinct groups: OEHB (OB), TEHB (TB), Full-transparent buffer (FT), and Pipeline (PL). Some types play a role in either nontransparent or transparent buffers as identified in previous research. Specifically:

- A nontransparent buffer typically consists of one slot of OB, one slot of TB, and $N - 2$ slots of FT, assuming N slots in total.
- A transparent buffer is comprised entirely of FT.

However, traditional configurations of these buffers are not mandatory. For instance, a channel requiring a latency of N could alternatively utilize a chain of N OB slots instead of a typical non-transparent buffer with the same number of slots. Given that OB generally has a lower area cost than FT and TB, this alternative can maintain the required throughput while reducing the overall area cost.

Furthermore, it is possible to mix different types of buffers within a single channel, provided they meet the circuit’s timing and throughput constraints, thus optimizing area cost. This flexibility is significant, especially in designs where different buffer configurations can lead to cost savings without performance loss.

To fully understand the impact of each buffer type on the circuit’s timing and throughput, it is crucial to examine how each buffer’s minimum and maximum latencies are determined. All buffer types must satisfy the constraint: $L_{min} \leq L_b \leq L_{max}$.

In subsequent sections, we will explore the specific conditions under which each buffer type operates, focusing on their respective L_{min} and L_{max} settings.

3.2 Minimum and Maximum Latency Analysis

In this section, we explore the minimum and maximum latency representations for each of the four types of buffers.

OB

OBs cut the timing paths of both the Valid and Data signals. For an OB slot, each token requires at least one clock cycle to reside before it can leave. Since a slot can accommodate only one token at a time, when the next token enters, the previous one must exit. Therefore, the II is the maximum number of cycles a token can stay in an OB slot. We use the latency (denoted by L) to represent the number of clock cycles a token stays in a single OB slot. The range of L can be expressed by the following constraint.

$$1 \leq L \leq II \quad (1)$$

For a multiple-slot OB chain, it can be considered equivalent to several single-slot OB channels cascaded together. The overall latency of the chain is the linear sum of the latencies of each individual channel, and naturally, their maximum and minimum values can also be summed. Therefore, for an N -slot OB, we derive the following constraint:

$$N \leq L \leq N \cdot II \quad (2)$$

To align with the existing optimization framework from previous work, we express this constraint in terms of the circuit's throughput (denoted by Θ) and the channel's token occupancy (denoted by $\dot{\Theta}$). The throughput of the circuit is the reciprocal of the II , and since all buffers in a dataflow circuit essentially act as queues, they conform to Little's Law. According to Little's Law, the average number of tokens in a buffer equals the average token entry rate multiplied by the average token residence time in the buffer. Thus, we have the following equations to represent the throughput and token occupancy:

$$\begin{cases} \Theta = \frac{1}{II} \\ \dot{\Theta} = \frac{1}{II} \cdot L \end{cases} \quad (3)$$

We combine equation (2) with equation (3) to derive the constraint for a channel with an N -slot OB concerning token occupancy instead of latency:

$$N \cdot \Theta \leq \dot{\Theta} \leq N \quad (4)$$

TB

TBs cut the timing paths of the Ready signal. Since each token can pass directly through TB, the minimum latency of a TB slot is 0. If a token stays in a TB slot, the next token, whether it enters this slot or passes through it, must wait at least until one cycle after the residing token has left. Therefore, the maximum latency of a TB slot is $II - 1$. Hence, we obtain the following constraint:

$$0 \leq L \leq II - 1 \quad (5)$$

Through the same derivation as before, we obtain the constraints for a channel with an N -slot TB regarding token occupancy:

$$0 \leq \dot{\Theta} \leq N \cdot (1 - \Theta) \quad (6)$$

FT

FTs do not cut any timing paths. Since an FT slot allows a token to pass directly through it and permits the exit of one token and the entry of the next one in the same cycle, we derive the range for its latency:

$$0 \leq L \leq II \quad (7)$$

Through the same derivation as before, we obtain the constraints for a channel with an N -slot FT regarding token occupancy:

$$0 \leq \dot{\Theta} \leq N \quad (8)$$

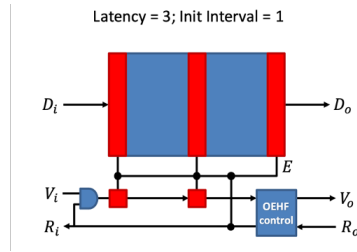
PL

Figure 2: PL Structure

The PL is a special type of buffer. As illustrated in Figure 2, a PL is composed of multiple slots controlled by a single OB control unit. A key operational difference between an M -slot PL and an M -slot OB is in token movement. Specifically, if the token in the last slot, which is nearest to the exit, doesn't leave during the current clock cycle, the entire PL stalls. This stall prevents tokens in other slots from advancing, even if their subsequent slots are unoccupied.

Aside from the stalling mechanism, the PL functions similarly to an OB. A token requires at least M clock periods to travel from the entry to the last slot of an M -slot PL. Throughout this duration, subsequent tokens may enter the PL due to the II of the entire circuit. After then, if another token enters the PL, the token in the last slot must leave the PL during the same clock cycle. Otherwise, the token about to enter the PL will be stalled, leading to a reduction in the throughput of the whole circuit. Experiments show that under a steady II, the token about to enter the PL is always the $\lceil \frac{M}{II} \rceil$ -th token after the one about to leave the PL. Based on it, we derive the range of the latency of an M -slot PL:

$$M \leq L \leq \lceil \frac{M}{II} \rceil \cdot II \quad (9)$$

Through the same derivation as before, we obtain the constraints for a channel with an M -slot PL regarding token occupancy:

$$M \cdot \Theta \leq \dot{\Theta} \leq \lceil M \cdot \Theta \rceil \quad (10)$$

The PL is special. Unlike other types, an M -slot PL is not composed of M -cascaded 1-slot PLs but functions as a single unit. While it is possible to cascade PLs, the maximum and

minimum latency cannot be linearly summed according to the slot number, due to the non-uniform structure within the PL. This structure allows for any number of slots to be connected to a single OB control unit. Therefore, it is appropriate to categorize PLs by defining each slot number as a specific type. That is to say, an M -slot PL governed by one OB control unit plays a similar role as a 1-slot OB, TB, or FT. Hence, when cascading N of M -slot PLs to create an $N * M$ -slot buffer, the linear sum of maximum and minimum token occupancy still applies:

$$N \cdot M \cdot \Theta \leq \dot{\Theta} \leq N \cdot \lceil M \cdot \Theta \rceil \quad (11)$$

In addition to chaining one type of buffer, the maximum and minimum token occupancy for the placement of mixed types of buffers in a single channel can also be linearly summed.

This concludes the description of the latency and token occupancy bounds for these four types of buffers. In the following section, we incorporate these constraints into an optimization model as an important component. The model aims to maximize throughput and minimize buffer area costs.

3.3 MILP Model for Performance and Area Optimization

We formulate our performance and area optimization model as a mixed-integer linear programming (MILP) model that determines the buffer placement strategy of every channel of the circuit, including buffer types and the size of each type. The model is based on the work of [1]. While retaining the same approach for handling timing constraints, we have integrated the token occupancy range modeling of buffers from the previous section into the throughput constraints, enabling the model to support analysis of the new buffer types we proposed.

We class constants and variables of the MILP model into three groups: input constants (i.e., values given as input to the MILP), output variables (i.e., the solution of the buffer placement problem), and internal variables (i.e., intermediate values found by the MILP solver but of little consequence to the user).

Input constants of the model:

- P (integer): Target clock period (CP) of the circuit.
- $P_{\max}$ (integer): Upper bound on the CP of the circuit, which has to be at least as large as any possible value of P .
- B_c (binary): Indicates whether channel c is a back edge ($B_c = 1$) of the dataflow graph (DFG).
- $D_{u,s}$ (real): Combinational delay of unit u for signal type s .
- $V_{i,s}$ (binary): Indicates whether buffers of type i cut the timing path of signal type s (True if $V_{i,s} = 1$).
- A_i (real): Indicate the area cost of a buffer unit of type i .
- w_d (real): The weight of the throughput of CFDFC d , which is proportional to the frequency of execution of CFDFC d .

Output variables of the model:

- $N_{i,c}$ (integer): The unit number of buffers of type i on channel c . For OB/TB/FTs, it indicates the slot number. For M -slot PLs, every M slots form a unit.

Internal variables of the model:

- Θ_d (real): Throughput of CFDFC d , ranging from 0 to 1.
- $\dot{\Theta}_{d,c}$ (real): Average token occupancy of channel c in CFDFC d .
- $r_{d,u}$ (real): Fluid retiming of tokens across unit u in CFDFC d .
- $R_{c,s}$ (binary): Indicates whether the buffer placement on channel c cuts the timing path of signal s .
- $t_{c,s}^{\text{in}}$ (real): Arrival time of signal s at the input of channel c (i.e., output of unit x , where $x \xrightarrow{c} y$).
- $t_{c,s}^{\text{out}}$ (real): Arrival time of signal s at the output of the channel c (i.e., input of unit y , where $x \xrightarrow{c} y$).
- $m_{d,c}^{\text{min}}$ (real): Lower bound of token occupancy of channel c in CFDFC d .
- $m_{d,c}^{\text{max}}$ (real): Upper bound of token occupancy of channel c in CFDFC d .

Then, we describe the constraints of the MILP model. We divide them into timing and throughput constraints.

Timing constraints

These constraints ensure that the entire circuit meets the target CP. They are therefore applied to the complete DFG. For each channel c of the DFG,

$$t_{c,s}^{\text{out}} \geq t_{c,s}^{\text{in}} - P_{\text{max}} \cdot R_{c,s} \quad (12)$$

with $t_{c,s}^{\text{out}} \geq 0$. This constraint propagates the combinational arrival time at each channel. In case of the presence of a buffer which cuts the specific signal path, the right term is guaranteed to be negative and $t_{c,s}^{\text{out}}$ becomes 0, essentially disabling the further accumulation of delays through this channel. The constraint requires an upper bound of the maximum cycle period (P_{max}).

The following constraints ensure that whenever it's necessary to cut the timing path of a specific signal type in a channel, there is a buffer within that channel capable of performing this cut.

$$\sum_i V_{i,s} \cdot N_{i,c} \geq R_{c,s} \quad (13)$$

The following constraints model the propagation delay of each unit u of the DFG with a pair of input/output channels $x \xrightarrow{c1} u \xrightarrow{c2} y$, for each signal type s .

$$P \geq t_{c2,s}^{\text{in}} \geq t_{c1,s}^{\text{out}} + D_{u,s} \quad (14)$$

The leftmost constraint enforces all delays to meet the cycle period P . Here, for simplicity, we assume that channels and buffers have zero delays. If we take into account delays within the buffers, the right-hand side of Equation (12) and the middle term of Equation (14) should include the relevant buffer delays. These delays are associated with the number of buffer slots and the sequence of buffer types in the placement.

Throughput constraints

Throughput constraints link steady-state token occupancy with circuit performance; we have augmented these constraints from prior work [1] with our buffer performance model, as discussed in Section 3.2.

In our model, throughput constraints are applied to individual CFDFCs rather than the entire DFG because even the same nodes and channels function distinctively under different CFDFCs. Following the approach of previous works, fluid retiming variables are assigned to each node of each CFDFC, and the token occupancy of each channel is determined by the difference in the retiming variables between its end and start nodes. For loops in a CFDFC, given that only one token can exist on a loop at any time, the sum of the token occupancy for all channels in a loop must equal 1. We initialize this token on the back edge of each loop, and subsequently, its occupancy is distributed to the channels of the loop by the retiming variables. Thus, we derive the following equation:

$$\dot{\Theta}_{d,c} = B_c + r_{d,v} - r_{d,u} \quad (15)$$

This method necessitates identifying the back edge of the loop before optimizing the model. Here, we clearly define the loop and the relationship between loops and back edges.

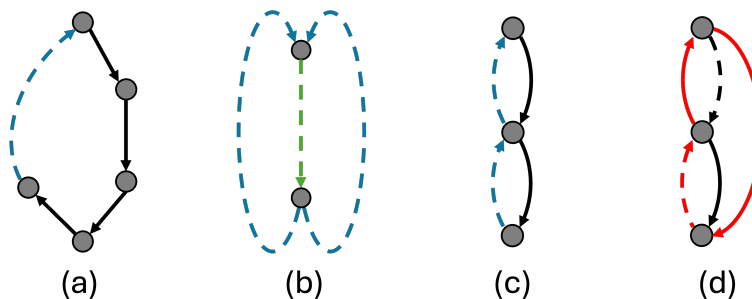


Figure 3: Possible Loop Structures

Definition 1: Each loop is an elementary cycle, a directed cycle in which each vertex, except for the start and end vertex (which are the same), is visited exactly once without any extraneous vertices or edges. Every loop contains exactly one back edge.

Figure 3 shows four examples of how we apply the definition. Figure 3(a) depicts a simple loop where any edge can be chosen as the back edge. Figure 3(b) illustrates two loops sharing a common channel, with two potential back edges indicated by blue dashed lines or one back edge shown by a green dashed line, both conforming to our definition. Figure 3(c) shows two vertically connected loops. Although a path can traverse both loops without repeating any edges, nodes

cannot be revisited, thus the larger loop does not exist. Consequently, we are free to use the two blue dashed lines to represent the back edges, without concern that a non-existent larger loop would have two back edges. In Figure 3(d), there are two vertically connected loops and a larger loop indicated by a red line, which also fits our definition of a loop. If we use the channels marked by the red line in the two smaller loops as back edges, the larger loop would exceed one back edge. Therefore, the channels marked by two dashed lines can be designated as back edges.

The definition helps select the appropriate edge as the back edge of a loop, ensuring that every loop within a CFDFC has a unique back edge. This is critical for our retiming constraints to function effectively.

Then, we delineate the relationship between each channel's token occupancy and its buffer placement. As noted in the previous chapter, once a channel has a specific buffer placement, its token occupancy is confined within a certain range. This range is influenced by the type of buffer occupying the channel and may also relate to the throughput of each CFDFC that includes this channel. If a channel exists within multiple CFDFCs, its buffer placement must satisfy the throughput constraints from all CFDFCs at the same time. To make it clear, we first set variables $m_{d,c}^{\min}$ and $m_{d,c}^{\max}$ denoting the bounds of the token occupancy, then describe them in formulas:

$$m_{d,c}^{\min} \leq \dot{\Theta}_{d,c} \leq m_{d,c}^{\max} \quad (16)$$

$$\begin{cases} m_{d,c}^{\min} = \sum_i f_i^{\min}(N_{i,c}, \Theta_d) \\ m_{d,c}^{\max} = \sum_i f_i^{\max}(N_{i,c}, \Theta_d) \end{cases} \quad (17)$$

Here, the functions $f_i^{\min}$ and $f_i^{\max}$ are predefined based on the specific types of buffers of interest. Below, we outline the four previously discussed types, according to (4), (6), (8) and (11). However, additional types, if developed and intended for implementation, can be incorporated in a similar manner.

$$f_i^{\min}(N_{i,c}, \Theta_d) = \begin{cases} N_{i,c} \cdot \Theta_d, & i = \text{OB} \\ 0, & i = \text{TB} \\ 0, & i = \text{FT} \\ N_{i,c} \cdot M \cdot \Theta_d, & i = \text{M-slot PL} \end{cases} \quad (18)$$

$$f_i^{\max}(N_{i,c}, \Theta_d) = \begin{cases} N_{i,c}, & i = \text{OB} \\ N_{i,c} \cdot (1 - \Theta_d), & i = \text{TB} \\ N_{i,c}, & i = \text{FT} \\ N_{i,c} \cdot \lceil M \cdot \Theta_d \rceil, & i = \text{M-slot PL} \end{cases} \quad (19)$$

The ceiling function can be linearized by assigning an integer variable that is less than or equal to it, and then substituting this function with the integer plus one. Thus, the model fundamentally remains an MILP model.

Objective

Subject to the timing and the throughput constraints, we maximize the weighted throughputs of all CFDFCs for a given CP P while accounting for the minimization of the total area cost of

buffers in the channels of the dataflow circuit.

$$\max : \sum_d w_d \cdot \Theta_d - \lambda \cdot \sum_{i,c} A_i \cdot N_{i,c} \quad (20)$$

where the weight w_d of each throughput is proportional to the frequency of execution of each CFDFC. λ is a small coefficient that gives a lower priority to the minimization of buffer area cost.

Although our modeling sets the buffer's area cost as a linearized value, in actual simulations, it is nonlinear. Since the area cost in this optimization model only appears completely in the objective, we can solve this issue by replacing the second term of the objective with the expression obtained from simulations, without affecting the usability of the model.

Modeling Units with Intrinsic Latency

Units with intrinsic latency are frequently used in HLS, such as pipelined computational units to tackle with very long delays. These units are able to cut part or all of the signal timing paths and accommodate tokens. So we can represent such a unit as two nodes connected by an internal channel with a buffer on it. For example, a unit with a latency of N , which cuts Data and Valid signal paths and contains at most N tokens, can be modeled as an N -slot OB or an N -slot PL placed on its internal channel between two nodes, depending on whether the last slot stalls the whole unit. If no specific buffer type adequately represents the behavior of such a unit, we can also characterize it through specific throughput constraints and its $V_{i,s}$ variables. In the subsequent discussion of variable units, we hypothetically use buffers to represent them.

With this, we have completed our description of the MILP model. Although this model considers units with intrinsic latencies, it still assumes that the latencies of these units are fixed. In actual circuits, there are units whose latency varies during operation, as well as units with variable II. Our current model cannot address the effects of these units; therefore, in the next chapter, we will discuss how to develop an improved model and how to simplify it.

4 Optimization Model for Variable Units

The MILP model described in Section 3.3 assumes a fixed latency and a constant initiation interval for each unit. However, in real-world applications, these parameters may become variable. We need to consider units that have a variable initiation interval, denoted as II_u , whose range is denoted by $[II_{u,min}, II_{u,max}]$. A common example of units demonstrating this behavior includes the read and write ports associated with a Load-Store Queue (LSQ). The LSQ plays a critical role in ensuring that memory accesses are executed in the correct sequence, adhering to the dependencies outlined in the program. This might lead to fluctuating rates at which the LSQ issues and retrieves data from the memory ports, thereby altering their effective II.

We also discuss computational units with variable latency such as telescopic units [2]. These units may take different amounts of clock cycles to compute results based on the input data. We denote their latency by L_u within the range of $[L_{u,min}, L_{u,max}]$.

In this discussion, we also consider the units performing both variable latency and variable II. Our goal is to *achieve the maximum possible throughput under any case of latency and II of each unit within their range, without changing the buffer placement.*

Previous work [1] summarized a straightforward approach to managing variable latency and II with a simple example. They identified a worst-case scenario such that if this scenario is satisfied, then all other cases would also be addressed adequately. In fact, this worst-case scenario always necessitates the largest buffer size because the transparent buffers used in their analysis—equivalent to the FT in this study—always meet the minimum token occupancy requirement, set at zero. Therefore, the primary requirement is to ensure the buffer size is large enough to satisfy the highest upper bound across all scenarios, leading to the largest buffer sizes. However, in the context of our study, where buffer elasticity is reduced, the minimum token occupancy is no longer assured due to the presence of OB and PL. The previous approach is unsuitable under these new conditions.

In the following, we describe 3 examples where the previous approach is inadequate in avoiding performance drop in the presence of latency/II variation. It is worth noting that although our previous MILP model primarily relied on CFDFC throughput and channel token occupancy to describe throughput constraints, we will use the II of CFDFCs and channel latencies in the following analysis, as they are more intuitive. The relationships between these two groups of variables are shown in Equation (3).

Case 1: Figure 4(a) illustrates a pair of reconvergent paths, which could be a subgraph of a CFDFC. On the left path, OP is a variable latency unit with latency ranging from 3 to 6. The right path features an OB, the size of which depends on our strategy. The II of the CFDFC, denoted by II_d , is assumed to be 1. According to the principles of retiming, the latencies of both paths should always be equal. If we represent OP with an N -slot OB when its latency is N , then buffer sizing becomes infeasible: According to Equation (2), since the II of the CFDFC is 1, there's only one value to choose as the latency, which is N . When the latency of OP is 6, the OB size should be 6; when the latency decreases to 3, the OB size should also be 3. This scenario contradicts the principle of fixed buffer placement. The only solution is to use a buffer type that has a more elastic token occupancy constraint, e.g., 3-slot OB followed by a 3-slot FT.

However, previous work considers only the maximum latency of OP when it is on a non-cyclic

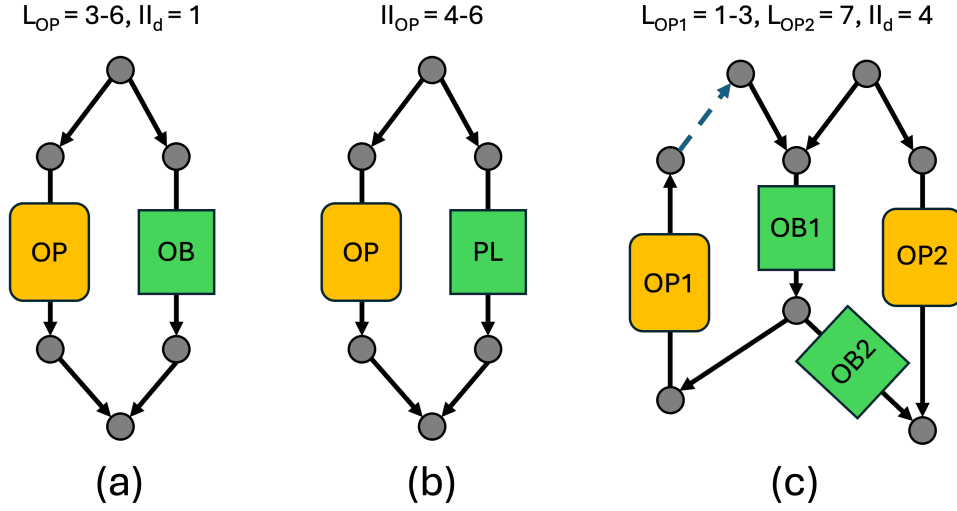


Figure 4: Problematic Cases

path, resulting in a 6-slot OB placement, which can lead to violations when the latency decreases to 3. Nevertheless, this problem can be resolved if the unit is modeled such that the slot number remains at its maximum. For instance, we could model the unit as an N -slot OB combined with a $\max(N) - N$ slot FT. We will adhere to this model in the subsequent analysis.

Case 2: Figure 4(b) is also a pair of reconvergent paths, OP on the left path is a variable II unit with the latency of 16. The right path features an N -slot PL, the size of which depends on our strategy. The II of the CFDFC depends on the II of OP. The latencies of both paths should always be equal. According to Equation (9), when the II of OP is 4, the minimum size of the PL could be 13 slots, since the ceiling function would increase its upper bound of latency to 16. However, if the II increases to 5, a 13-slot PL would no longer meet the constraints because the ceiling function would only achieve the maximum latency of 15, which is insufficient. Previous work have typically considered only the minimum II as the worst case; however, the placement strategy that works for the minimum II does not necessarily suffice when the II is higher. Additionally, the strategy becomes viable again when the II reaches 6. Therefore, neither the minimum nor the maximum II can ensure compliance under all scenarios.

Case 3: Figure 4(c) depicts a loop whose right side is on a pair of reconvergent paths. OP1 on the loop is a variable latency unit with latency ranging from 1 to 3. OP2 is a unit with a latency of 7. The sizes of OB1 and OB2 depend on our strategy. The II of the CFDFC is 4, determined by another part of the CFDFC (not shown in this figure). Our sizing strategy should ensure that the latencies of OB1 and OB2 together at least equal 7, matching OP2's latency, and that the latencies of OB1 and OP1 together equal 4, matching the II of the CFDFC. According to the strategy of previous work, we should consider the minimum latency of OP1 as the worst case since it is on a cyclic path. In this case, with OP1's latency at 1, we deduce that the latencies of OB1 and OB2 are 3 and 4, respectively. Both OBs need only 1 slot. However, when OP1's latency increases to 3, then the latency of OB1 should only be 1 to prevent an increase in the II,

which would decrease throughput. Consequently, OB2's latency must be 6, which exceeds the upper bound for a 1-slot OB under an II of 4. Therefore, we need to initially assign 2 slots to OB2 to avoid this problem.

In theory, we have to incorporate all possible values of the latency/II that the units can take in the MILP model; this guarantees that the placed buffers satisfy the throughput constraints (see Equation (16)-(19)) in all possible cases. We show that we can identify a significantly reduced set of scenarios (i.e., combinations of latency/II values) that would be sufficient to consider when building the MILP model.

In the following sections of this chapter, we will discuss specific conditions that can be exploited to reduce the number of scenarios which are necessary to be considered. Ultimately, we will modify the previous MILP to propose a formal optimization flow that considers multiple scenarios simultaneously.

4.1 Basis of the Analysis

Since our goal is to achieve the maximum possible throughput for each scenario, we first need to determine its value; otherwise, we cannot confirm whether we have achieved the maximum possible throughput. In order to streamline the optimization model while achieving our goal, we will proceed with the following logical analysis: After determining the maximum possible throughput for each scenario, if the buffer placement optimized for one scenario not only satisfies the maximum possible throughput for that scenario but also for another, then it is unnecessary to include the latter scenario in the optimization model. By employing this method, we will develop a model that considers as few scenarios as possible, thereby reducing the runtime cost of solving the model.

Reinterpreting the Throughput Constraints

Before determining each maximum possible throughput, there is still some preliminary work to be done. Since we use latency of channels and II of the CFDFC to characterize the problem in our subsequent analysis, we first need to reinterpret how throughput constraints manifest under these variables.

In Equation (15), the token occupancy of each channel is determined by the difference in retiming between the end and start nodes plus B_c . According to *Definition 1*, in a CFDFC, each loop contains exactly one back edge. As the B_c term for back edges in Equation (15) is set to 1 and 0 for others, and since the total retiming difference for a loop is zero, the sum of token occupancies for channels in any loop equals 1. This implies that the sum of buffer latencies for any loop in the CFDFC during operation equals the II of the CFDFC, representing the first type of constraints conveyed by this formula, which we refer to as *loop constraints*. In Figure 5(a), the paths 1-2-3-4-5 and 6-7-8-4-5 exemplify such loops.

Additionally, for paths that are not back edges, if there are multiple paths (referred to as reconvergent paths) between two nodes, the sum of token occupancies for channels on each path should be equal to the retiming difference between the two nodes. Their latency sum is also equal, forming the second constraint conveyed by this formula, which has already been utilized earlier. However, in the CFDFC graph, these reconvergent path pairs are not necessarily independent.

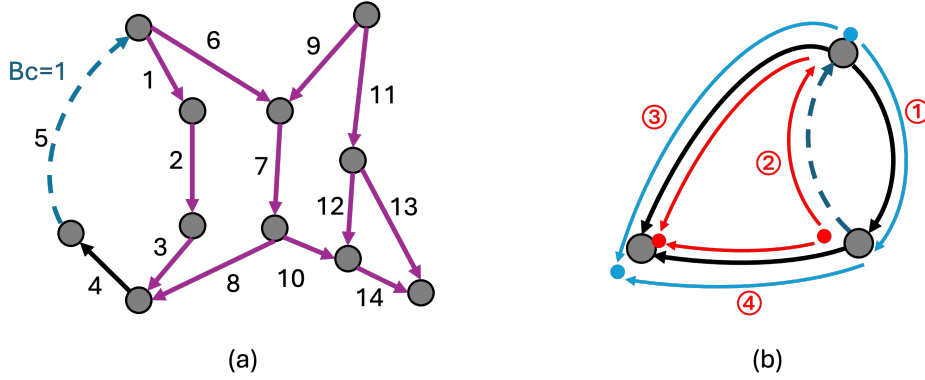


Figure 5: Throughput Constraints on CFDFC

In a CFDFC with variable latency and II units, if a change in one latency/II scenario leads to a modification in the token occupancy of a channel, this change affects all reconvergent path pairs associated with that channel. This may cause changes in other channels' token occupancies, potentially involving more reconvergent path pairs, leading to far-reaching effects and eventually forming a closed, independent network of reconvergent paths. In Figure 5(a), such a network is illustrated in purple. Paths 1-2-3 and 6-7-8 are a pair of reconvergent paths. Changes in token occupancy of channel 7 could impact the constraints on the path pair 9-7-10 and 11-12, possibly leading to changes in token occupancy of channel 12, thereby affecting the constraints on the path pair 12-14 and 13, ultimately forming such a network. We refer to the constraints within the network as *reconvergent path constraints*.

Finding 1: We can safely exclude every back edge from reconvergent path networks.

As shown in Figure 5(b) as an example, channel ①-② forms a loop, where channel ② is the back edge. Path ②-③ and ④ constitute a reconvergent path pair, as indicated by the red arrows. Using the channel number to represent the token occupancy of each channel, and considering that the B_c of the back edge is 1, the relationship can be expressed by the equation $② + ③ + 1 = ④$. Also, the loop constraint shows that $① + ② = 1$. Integrating these two equations yields $③ = ① + ④$, as depicted by the blue arrows in the figure. Thus, the reconvergent path constraint on the back edge is transferred to the channels of the loop excluding the back edge. Subsequently, the set of reconvergent path constraints involving back edges becomes redundant, and it is sufficient to consider only the loop constraint for the back edge, which can then be excluded from the reconvergent path network. Although this example does not represent all cases, the method of transferring constraints from the back edge to other paths through the loop constraint is generally applicable.

To identify reconvergent path networks and their constituent channels, we propose the following method: First, remove the back edges from the CFDFC. Next, convert the CFDFC into an undirected graph. Each biconnected component within this undirected graph represents a reconvergent path network. Biconnected components can be identified using graph algorithms such as Tarjan's algorithm[3].

In addition to these two constraints, we also need to adjust the buffer capacity-related through-

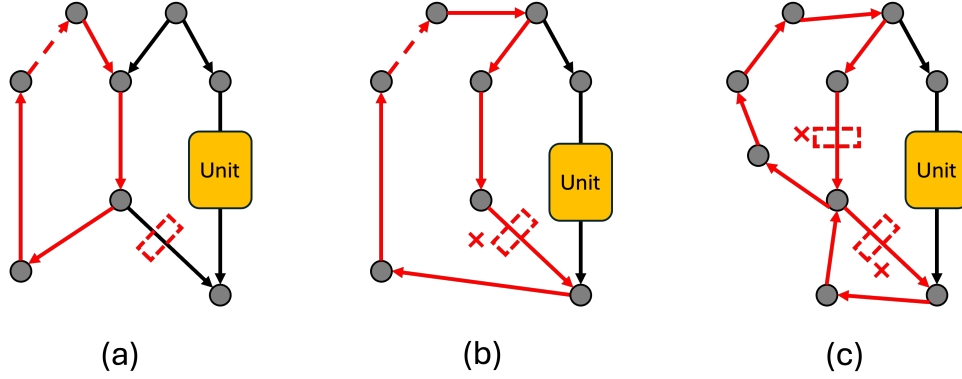


Figure 6: Units not on the Loop Do not Affect Throughput

put constraints to align with latency and II formats, as previously discussed in the text. These three forms of constraints together constitute the reinterpreted throughput constraints.

The Maximum Possible Throughput

After discussing the reinterpreted throughput constraints, we explore what the actual value of the maximum possible throughput is for each scenario.

Across scenarios, the variables related to timing constraints in MILP remain invariant. After applying timing constraints to the entire DFG to determine $R_{c,s}$ for all channels and signals, we establish the upper bound of throughput for a CFDFC in each scenario. Specifically,

$$\Theta_d^{max} = 1/\max\{(\sum_{c \in loop} R_{c,Data} + \sum_{u \in loop} L_u) \forall loop, II_u \forall u\} \quad (21)$$

The terms in the max function represent the fundamental restrictions on the throughput of the CFDFC. The loop related terms are similar to the cycle time of the CFDFC [4]. Within each loop of the CFDFC, we cannot avoid the intrinsic latency of units or the latency of $R_{c,Data} = 1$ channels, where we are required to place buffers that add at least one latency on the Data signal path. For variable latency units, L_u varies with each scenario. For other units with intrinsic latency, the L_u remains constant. For units without any latency, L_u is zero. The loop with the highest total latency due to these unavoidable latencies becomes one of the factors limiting throughput. Another throughput-limiting factor is the II_u of variable II units, as it is possible that II_u may further restrict throughput beyond what is imposed by loops.

After we establish the upper bound of throughput for each scenario, we still need to demonstrate that it can be consistently achieved across all scenarios through a fixed buffer placement. Otherwise, we cannot demonstrate that this is the maximum possible throughput.

Finding 2: There always exists a fixed buffer placement which can achieve the maximum throughput stated in Equation (21) under all scenarios.

Here, we propose a buffer placement that meets the requirements. By adding a 1-slot OB to every $R_{c,Data} = 1$ and $R_{c,Valid} = 1$ channel, and a 1-slot TB to every $R_{c,Ready} = 1$ channel,

and by placing infinite FT slots (or large enough, if we want it to be finite) on all channels, the constraint Equation (13) is satisfied and the latency lower bounds of the buffers comply with the unavoidable latencies induced by $R_{c,Data} = 1$, while the upper bounds become infinite. Thus, if we specify the CFDFC throughput according to Equation (21), the throughput constraints related to buffer capacity are always met. The only remaining concern is whether the combination of reconvergent path constraints and loop constraints increases the total latency of the largest loop. Referring to Figure 6(a) & (b) depicting a loop on the left overlapped with a reconvergent path pair on the right, we assume that the loop shown in red is the largest. In Figure 6(a), if the rightmost path has a significantly large unavoidable latency—possibly due to a high-latency unit or a very long timing delay—that exceeds the unavoidable latency of its left reconvergent path, then the left reconvergent path must increase its actual latency when the circuit is operational, to align with the rightmost path. Under this condition, we could find a channel not on any loop, such as the one marked by a red dashed rectangle, to absorb the extra latency without increasing the latency of channels on the loop. In Figure 6(b), in a similar situation but we cannot find a channel not on any loop to absorb the extra latency, an increase in latency seems to be unavoidable. However, we observe that the leftmost part of the loop, which is not part of the reconvergent path pair, can be connected with the rightmost path to form an even larger loop, contradicting the assumption that the loop previously considered was the largest. In Figure 6(c), where we assume multiple largest loops share the same reconvergent path. This situation is similar to the structure shown in Figure 3(d). By connecting all the leftmost parts of each loop with the rightmost path in the figure, we identified a larger loop, which contradicts our previous assumptions. Thus, the largest loop can maintain its latency at the unavoidable level, unaffected by loop and reconvergent path constraints. In conclusion, by using an extreme strategy, we identify a buffer placement that achieves maximum throughput under all scenarios, thereby proving that such optimization is always attainable.

In conclusion, Equation (21) represents the 'maximum possible throughput' required for our analysis. In the following sections, we will discuss from the perspectives of II and latency which scenarios can be satisfied by the outcomes obtained from other scenarios to meet the maximum possible throughput, thus rendering them unnecessary for consideration.

4.2 Variable Initiation Interval

This section discusses whether a given buffer placement can adapt to changes in II when the latency of all variable units is specified. By proving our finding, under certain circumstance, optimizing a CFDFC with variable II is equivalent to optimizing a CFDFC with fixed II values.

Here, the II of individual variable units does not necessarily represent the II of the CFDFC. The II of the CFDFC is determined based on the maximum value among the latency of all its loops and II specified for each unit. When the II of a unit becomes the maximum value, an increase in it will lead to an increase in the overall CFDFC's II; otherwise, it has no impact on the CFDFC, and this scenario can be disregarded.

Definition 2: For a CFDFC, if the latency upper bounds of all types of buffers placed on its channels as well as all the units with intrinsic latency increase at least proportionally to the

increase in II, specifically:

$$\frac{\Delta L_{max}}{\Delta II} \geq 1 \quad (22)$$

and the latency upper bound of each separable buffer slot is no larger than II, we call it II-elastic CFDFC. For the four types of buffer we consider currently, only PL doesn't apply to this requirement.

Finding 3: For an II-elastic CFDFC of any structure, once its buffer placement is established for a given value of II, this configuration is capable of supporting any larger II without modification, given that latency of all variable units remain unchanged.

According to Equation (21), if the maximum possible throughput is solely determined by the largest loop, then an increase in the II of any variable II unit will not impact the throughput until this unit's II becomes dominant. In such cases, any increment of the dominant unit's II by 1 would increase the minimum possible II of the CFDFC, which corresponds to the reciprocal of the maximum possible throughput, by 1. To substantiate *Finding 3*, we need only demonstrate that after an increase of 1 in the CFDFC's II, the original buffer placement still meets the constraints under the new II. By extrapolation, this also confirms that any further increase in II would be similarly accommodated.

In fact, there is a direct proof derived from the equations in MILP. An increase in II only reduces the throughput variable Θ_d . It is possible to maintain unchanged token occupancy for each channel. For all considered buffer types except for PL, a decrease in Θ_d merely expands or maintains the token occupancy range, ensuring that if buffer capacity-related constraints were not violated in the original scenario, they would remain unviolated. Since the token occupancies remain unchanged, both loop and reconvergent path constraints continue to be satisfied. However, the range of PL token occupancy could be narrowed, which is why it is disallowed. Figure 4(b) serves as a sufficient counterexample.

While this proof holds under theoretical conditions, in an actual circuit, the token occupancy should be expressed as a fraction with II as the denominator, given that latency must be integral. Solutions from the previous proof, which maintain the denominator of token occupancy as the original II rather than the new II, result in non-integer latencies and cannot be precisely implemented in actual circuit behavior. Thus, a proof assuming integer latency still holds value. Here, we introduce a proof that assumes integer latency, which will also support some findings related to variable latency units in the subsequent section.

To prove the scenario when II increases by one, it is only necessary to identify a new set of retiming values for each node and token occupancy values for each channel (determined through its latency) so that all constraints are satisfied under the new II. For constraints related to buffer capacity, if the buffer latency of a channel was at its lower bound in the original scenario, then the new latency cannot be lower. If the buffer latency of a channel was at its upper bound, in accordance with the II-elastic characteristics of the CFDFC, the new latency can increase by at least one without exceeding its upper bound. For channels without buffers, their latency is zero, and the new latency must also remain zero.

Under the new II, we first initialize the latency of each channel to its original value, ensuring that constraints related to buffers and reconvergent paths continue to be met. For each loop, the sum of the latency across all channels needs to increase by one to achieve the new II value.

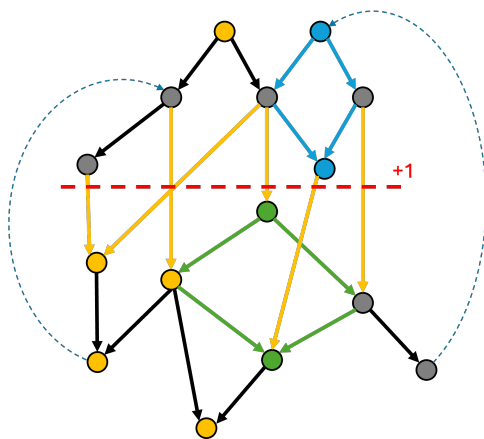


Figure 7: Traverse of a Reconvergent Path Network

This means that within each loop, only one channel's latency needs to increase by one, while the rest remain unchanged, since it's unsafe to decrease the latency of any channel due to buffer capacity constraints. When deciding which channel's latency to increase, care must be taken to adhere to reconvergent paths constraints; the number of channels with increased latency on each side of a pair of reconvergent paths must be equal.

To simplify the problem, we propose a traverse on the reconvergent path network. As illustrated in Figure 7, in a reconvergent path network, we first arrange the nodes based on the value of retiming; nodes with larger retiming values are positioned lower. We then select a value such that the retiming of nodes greater than this value is increased by the new throughput value, while the retiming of nodes below this value remains unchanged. This is the same as crossing the network with an isoline of retiming (illustrated as a red dashed line in the figure), increasing the latency of all channels crossed by this line (depicted in yellow) by one. For path pairs where both nodes are either above or below the isoline (shown in blue and green), this operation does not affect the original reconvergence constraints. For path pairs with nodes on opposite sides of the isoline (some nodes highlighted in yellow as examples), since there are never back edges in the network, the retiming values along any path are monotonically increasing. Thus, all paths between the two nodes cross the isoline exactly once, increasing the latency by one, and still satisfying the constraints. Moreover, since the retiming values of the two ends of each channel crossing the isoline are not equal, indicating that the token occupancy is not zero, buffer must be present on this channel, which can accommodate the increased latency. Therefore, we demonstrate that this operation, while increasing latency, does not violate the reconvergent path constraints.

Then, we propose a strategy for performing traverses on the CFDFC under the original Π , which changes its existing retiming and token occupancy to a solution that satisfies all constraints under the new Π , regardless of the CFDFC's structure. We first divide channels with multiple buffer slots into smaller channels each containing a single slot, a change that does not alter the graph's structure. According to *Definition 2*, since the latency upper bound of each separable buffer slot does not exceed the Π , the token occupancy of each channel will not exceed 1.

Our strategy for arranging the traverses starts with the node having the smallest retiming value, adding a value smaller than the original throughput as the first traverse value, and then continues incrementally by 1 until it surpasses the maximum retiming value of the CFDFC. Since each node's retiming can be expressed as a fraction with the original II as the denominator, the designed traverse values will avoid all node retimings, thus avoiding exact alignment with any node.

For each channel after decomposition, since its token occupancy will not exceed 1, it will at most undergo one traverse. Therefore, this channel's latency will either remain unchanged or increase by 1, in accordance with buffer capacity-related constraints.

For each loop in the CFDFC, we require the total latency sum to increase precisely by 1 to meet the new II. If a loop's back edge has no token occupancy, its retiming span is exactly 1. According to our strategy, this loop will undergo exactly one traverse, thus its latency increases precisely by 1. We reiterate here that traverse is applied to the reconvergent path network and does not affect the back edge.

If a loop's back edge has token occupancy, its retiming span will be less than 1, so it might undergo one traverse or none. If it undergoes no traverse, we can add 1 unit of latency on the back edge to meet the new II, since the presence of token occupancy on the back edge indicates the presence of a buffer, thus providing the elasticity to increase latency by 1. If multiple loops share the same back edge, their retiming ranges must be consistent, therefore they either all undergo a traverse or none do, maintaining the consistency of whether or not the back edge's latency is increased. Thus, we are assured of finding an existing solution to meet the increased II without altering the buffer placement. This concludes our demonstration of *Finding 3*.

To conclude this section, if there is no PL and any unit with intrinsic latency is not modeled as a PL, optimizing a CFDFC with variable II units is equivalent to optimizing it with all units' IIs fixed at their minimum values.

4.3 Variable Latency

This section discusses whether a given buffer placement can maintain the maximum possible throughput under changes in the latency of some variable latency units. Through two key findings, we illustrate that some scenarios do not need to be considered.

Here, the latency of individual variable units does not necessarily represent the actual latency of the unit when the CFDFC is running, but rather the minimum latency requirement of the unit. A larger latency is allowed as long as it does not lower the throughput of the CFDFC. For clarity, we define the minimum latency requirement for a variable unit as L_u , while the latency of the unit during actual circuit operation will still be referred to as latency.

Based on the conclusion of the previous section, all variable II units are fixed at their minimum II, and thus do not require excessive consideration. Given the complexity attributes of PL, the following discussion will only focus on II-elastic CFDFC.

Two Key Findings

This part presents and proves two key findings. We utilize them to eliminate certain scenarios among the various value combinations of variable latency units that do not need to be considered,

thereby simplifying the optimization model.

Finding 4: For a CFDFC with variable latency units, if a scenario has been optimized and reducing the latency of a variable latency unit does not affect the minimum possible II of the CFDFC, then it is unnecessary to optimize the scenario with the reduced latency.

According to our interpretation of Figure 4(a), when modeling variable latency units, we consistently maintain the number of slots at their maximum L_u . As a result, the latency upper bound of the unit always aligns with the scenario where L_u is at its maximum. This means that there is no need to reduce the actual latency for the unit in scenarios with a smaller L_u if we adhere to the buffer capacity constraints. Moreover, since the target II remains unchanged, there is no justification for decreasing the latency of the specified unit. If we ensure that all constraints are satisfied when L_u is at its maximum, the same optimization results will directly apply to scenarios with smaller L_u values, without requiring any modifications.

It is important to note that it is still desirable to minimize the latency of units on the loop currently dominating the maximum possible throughput, as reducing latency can decrease the minimum possible II and enhance the maximum possible throughput. Moreover, this finding does not hold in the opposite direction. Figure 4(c) serves as a counterexample, demonstrating that a scenario with a smaller L_u cannot adequately cover a scenario with a larger L_u , if the minimum possible II remains unchanged.

This condition applies not only to some variable latency units on the loop but also to all variable latency units not on the loop, as their latency has no impact on II. Although this is intuitive, for the sake of rigor, we will still demonstrate it through Figure 6 in a manner similar to how we previously proved that the maximum possible throughput can be achieved.

As shown in Figure 6(a), for any reconvergent path pair involving a variable latency unit, the potential disadvantage of increasing L_u is that if there is a channel on a loop on the opposite side, it may result in increased latency for that channel, thereby reducing the throughput of the loop. However, when optimizing for the scenario where L_u is maximized, we can insert more buffers on a channel in the opposite path, which is not on a loop, to increase the latency upper bound of the whole path, while minimizing the latency on the channel which is on the loop. Therefore, we do not need to reduce the unit's latency to achieve maximum throughput.

As depicted in Figure 6(b), if the entire other side of the reconvergent path pair belongs to a single loop, we cannot find a channel not on the loop to insert more buffers. In this case, by replacing the left path with the right path of the reconvergent path pair, we find a new loop, meaning that the unit is actually on a loop, which does not meet our assumption. As illustrated in Figure 6(c), if the other side of the reconvergent path pair is occupied by multiple loops, we cannot arbitrarily insert buffers. However, we can demonstrate that the unit is on a larger loop, which consists of the leftmost paths of each channel and the rightmost path in the figure.

Finding 5: For a CFDFC with variable latency units, if a scenario has been optimized and any increase of the latency of a variable latency unit will increase the minimum possible II of the CFDFC, then it is unnecessary to optimize the scenario with the increased latency.

This finding applies to units on the currently largest loop, which dominates the maximum possible throughput. For each increase of one in L_u , the entire CFDFC's II increases by one. This scenario is equivalent to that depicted in Section 4.2, where we discuss the traverse strategy to increase II of the whole CFDFC, with the additional constraint that the latency of the unit

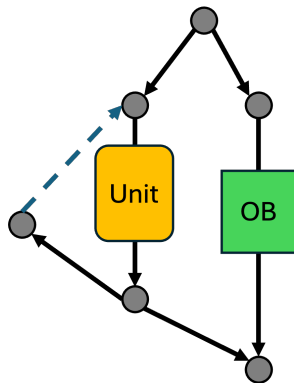


Figure 8: The Converse of Finding 5 is not True

itself must increase by 1. If the unit is on the back edge, then all its associated loops have a retiming span smaller than 1, and their minimum and maximum retiming values are identical since they share the same back edge. Due to the increase in the unit's latency, we shouldn't further traverse the associated loops. Thus, we first place a traverse slightly less than the minimal retiming value among the nodes of these loops. We then place other traverses at an interval of 1 across the entire retiming range of the CFDFC. According to our explanation in Section 4.2, the II of the CFDFC increases by exactly one after the traverses, with no constraints violated. Moreover, the loops associated with the unit do not undergo any traverse, complying with our additional constraint.

If the unit is not on the back edge, we must perform a traverse within the unit's retiming range to represent its increased latency, while the remaining traverses are still placed at intervals of 1 across the CFDFC, thus violating no constraints.

By extrapolation, this also confirms that any further increase in L_u would be similarly accommodated by repeating the process of increasing by 1. This concludes our proof of *Finding 5*.

Conversely, if in a certain scenario, a decrease in a unit's L_u leads to an increase in the maximum possible throughput that the CFDFC can achieve, then even if we have optimized the original scenario, we cannot overlook the altered scenario. We demonstrate this with a simple counterexample. As shown in Figure 8, a variable latency unit is located on a loop on the left and is part of a reconvergent path pair on the right. If this figure represents the entirety of a CFDFC, then naturally, the CFDFC's II is determined by the unit's latency, and any reduction in latency will impact the II. Suppose L_u is 3, then the actual minimum latency of the unit is 3, thus setting the CFDFC's II at 3. The optimization model might add a 3-slot OB to the right to satisfy the reconvergent path constraint, as OB is a cost-effective option. However, if the unit's L_u decreases to 2, the lower bound of the OB's latency ensures that the unit's actual latency remains at 3, thereby keeping the CFDFC's II at 3. Yet, in the optimal scenario, using a 3-slot FT or a combination of a 2-slot OB with a 1-slot FT can reduce the buffer's latency to 2, thereby lowering both the unit's latency and the CFDFC's II to 2.

To summarize this section, in deciding whether to consider optimizing a scenario with a

combination of variable latency values, we initially screen scenarios with all variable IIs set at their minimum (II-worst case). For a given scenario, if decreasing a unit's L_u by 1 does not change the CFDFC's maximum possible throughput, we disregard the altered scenario. Similarly, if increasing a unit's L_u by 1 reduces the CFDFC's maximum possible throughput, we also disregard the altered scenario. From this, we infer that if decreasing a unit's L_u by 1 increases the CFDFC's maximum possible throughput, we prioritize optimizing the altered scenario. Likewise, if increasing a unit's L_u by 1 does not affect the CFDFC's maximum possible throughput, we also prioritize optimizing the altered scenario. The final set of scenarios selected for optimization must satisfy both the following two properties:

- Increasing the L_u of any unit that has not yet reached its maximum by 1 will decrease the CFDFC's maximum possible throughput. This property holds if the L_u of every variable latency unit has reached its maximum;
- Decreasing the L_u of any unit that has not yet reached its minimum by 1 does not affect the CFDFC's maximum possible throughput. This property holds if the L_u of every variable latency unit has reached its minimum.

We can ensure that such a scenario exists. Assume no such scenario exists; starting from all units' $L_{u,min}$, the second property is already satisfied, thus there must be a unit whose L_u can be increased by 1 without affecting the maximum possible throughput to violate the first property. According to our summary, we prioritize optimizing the altered scenario. We then take this new scenario as the starting point for the next iteration. Such iterations will always maintain the maximum possible throughput unchanged. Since the scenario where all units are at $L_{u,min}$ represents the highest maximum possible throughput, the second property is always satisfied during iterations, and we must ensure that a unit that does not satisfy the first property is found to continue the iterations. Since each iteration inevitably increases the L_u of some unit, if the iterations never stop, when all units' L_u reach $L_{u,max}$, the first property will be satisfied, and thus we have identified a scenario requiring optimization. If the iterations stop midway, the scenario at which they stop is the scenario that needs to be optimized.

4.4 Divide a CFDFC to Regions

In this section, we introduce an important method to minimize the cost of the optimization model. By dividing a CFDFC into separate regions, we eliminate unnecessary relationships among them.

Finding 6: For a CFDFC, after transforming it into an undirected graph, we treat each biconnected component as a region. There's no cross-region reconvergent path constraints or loop constraints.

This approach is similar to our discussion on reconvergent path networks, and Tarjan's Algorithm is still applicable. As shown in Figure 9, the CFDFC is divided into three regions. The regions overlap at a maximum of one node. However, since retiming actually functions as a relative value rather than an absolute one, what really impacts buffer placement is the token occupancy of the channels, so the connection of only a node does not impose constraints on each other. This implies that each region can be treated as an independent CFDFC, allowing for

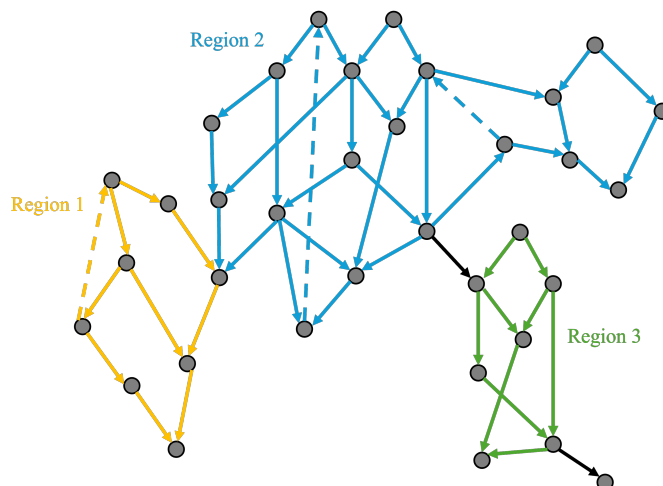


Figure 9: Divide a CFDFC into Regions

the analysis of its throughput constraints across scenarios involving its own variable units. For channels that are not part of any region, it is safe to apply no throughput constraints; however, timing constraints are still necessary. Therefore, the buffer placement for these channels will only be constrained by timing constraints.

Although we will not immediately explore this issue, this dividing method is very important for the upcoming optimization model. In the next section for optimization model, each scenario to be considered will require setting up a new set of retiming and token occupancy variables. Suppose Region 1 has 10 scenarios to consider and Region 2 has 20; if we treated them as a whole during optimization, we would have to multiply these scenarios, thus assigning 200 sets of variables for the entire CFDFC. However, by separating the regions, we only need to allocate 10 sets and 20 sets of variables for their respective regions, significantly simplifying the optimization model.

Since we treat each region as an independent CFDFC for analysis, the conclusions from previous sections are equally applicable to regions. When selecting the necessary scenarios for each region, we can directly use the criteria proposed in the summary of Section 4.3. Compared to filtering for the entire CFDFC, the method of dividing into regions significantly simplifies the selection process. However, there are still interconnections between the IIs of different regions. Since the II of each region within a CFDFC must be unified, the minimum possible II for each region should be determined by taking the maximum of the minimum possible IIs obtained from the separate analysis of each region. While this may seem to lead to complex combinations, it is unnecessary because, according to *Finding 2*, when analyzing one region, we need only consider the scenarios of the other regions that result in their lowest minimum possible II. To address this, we propose a definition.

Definition 3: For a CFDFC with variable units, the highest maximum possible throughput is termed the prime throughput, denoted as Θ_d^{prime} . The reciprocal of Θ_d^{prime} is termed the prime II, denoted as II_d^{prime} .

$$\begin{cases} II_d^{prime} = \max\{(\sum_{c \in loop} R_{c,Data} + \sum_{u \in loop} L_{u,min}) \forall loop, II_{u,min} \forall u\} \\ \Theta_d^{prime} = 1/II_d^{prime} \end{cases} \quad (23)$$

The minimum possible II of any region should not be lower than II_d^{prime} . If the minimum possible II for a scenario within a particular region falls below II_d^{prime} , then the minimum possible II for all scenarios in the remaining regions must be at least II_d^{prime} , thereby raising the minimum possible II for that region. Conversely, if the minimum possible II for a scenario within a region reaches or exceeds II_d^{prime} , then the other regions will not affect the II of that scenario in the given region.

To conclude this section, we identify the biconnected components of the CFDFC as regions. First, we calculate the overall II_d^{prime} for the CFDFC, ensuring that the II of each region is not less than this value. Subsequently, we treat each region as an independent CFDFC for analysis, selecting the necessary scenario sets to consider for subsequent optimization modeling.

4.5 A Comprehensive MILP for Variable Units

Previously, our MILP model could only optimize fixed L_u and II_u values. Now, we propose an expanded optimization model that considers multiple scenarios to ensure that the same buffer placement can achieve maximum throughput across all combinations of variable L_u and II_u for each CFDFC.

Since timing constraints and their corresponding variables, as well as those related to buffer placement, do not change across different scenarios, we establish these variables for all channels and nodes throughout the entire DFG. Variations in each CFDFC scenario lead to changes in variables related to throughput constraints, specifically, the retiming of each node, the token occupancy of each channel, and the throughput of the CFDFC. Previously, we proposed dividing the CFDFC into regions. Consequently, for each region's scenario, we need to establish a new set of throughput-related variables, incorporating the L_u and II_u in that region into the constraints and setting the throughput variable to the maximum possible throughput. This differs from our previous model, where throughput appeared as a variable. However, now we must first calculate the maximum possible throughput for each scenario based on the results of the timing constraints and then incorporate it as a constant into the constraints of each scenario. Here, we use a similar MILP model to compute the results with only timing constraints applied. Thus, the optimization process is divided into two steps. Algorithm 1 is the pseudocode describing the optimization process.

The algorithm is divided into two steps: lines 3-7 and lines 9-33. The first step, lines 3-7, aims to determine the distribution of $R_{c,s}$ to calculate the maximum possible throughput. Here, we only add variables and constraints related to timing across the DFG, as in line 3-4. It is crucial to minimize the number of timing breakpoints $R_{c,s}$ in line 5, as excessive number of them can impose unnecessary constraints on buffer placement, potentially reducing the achievable throughput. Once solved, line 7 saves R_c as a constant for use in the second step to calculate the maximum possible throughput.

The second step, lines 9-33, focuses on reducing the buffer area cost while consistently achieving maximum possible throughput under variable latency/II conditions. Lines 9-11 start by

Algorithm 1 Two Step Optimization for Variable Units

```

1: Load all constants
2: First-step MILP:
3: Set variables  $R_{c,s}, t_{c,s}^{in}, t_{c,s}^{out}$  for DFG
4: Set Equations (12), (13), (14) for variables
5: Set Obj.  $\min \sum_{c,s} R_{c,s}$ 
6: Solve MILP
7: Save  $R_{c,s}$  output as constants
8: Second-step MILP:
9: Load all constants including  $R_{c,s}$ 
10: Set variables  $N_{i,c}$ 
11: Set Equation (13) for  $N_{i,c}$ 
12: for CFDFC  $d$  do
13:    $\Theta_d^{prime} \leftarrow$  Equation (23)
14:    $L_{u,min} \leftarrow L_{u,max} \forall u$  not on the loop
15:   Establish regions
16:   for Region  $r$  do
17:     for Scenario  $s$  do
18:        $II_u \leftarrow 1/\Theta_d^{prime}$ 
19:        $\Theta_{d,r,s} \leftarrow$  Equation(21)
20:       Build set  $inc_s, dec_s$ 
21:     end for
22:     for Scenario  $s$  do
23:       if ( $\Theta_{d,r,s'} < \Theta_{d,r,s} \forall s'$  in  $inc_s$  or  $|inc_s| = 0$ )
         and ( $\Theta_{d,r,s'} = \Theta_{d,r,s} \forall s'$  in  $dec_s$  or  $|dec_s| = 0$ ) then
24:         Set variables  $\dot{\Theta}_{d,c,s}, r_{d,u,s}, m_{d,c,s}^{min}, m_{d,c,s}^{max}$  for region  $s$ 
25:         Load  $\Theta_{d,r,s}$  as a constant
26:         Set Equations (15), (16), (17), (18), (19) for related variables
27:       end if
28:     end for
29:   end for
30: end for
31: Set Obj.  $\min \sum Area$ 
32: Solve MILP
33: Save  $N_{i,c}$  output
  
```

adding variables for buffer types and sizes across the entire DFG and integrating $R_{c,s}$ related constraints for buffers. Lines 12-30 involve adding throughput variables and constraints by iterating through scenarios in each region of each CFDFC. For each CFDFC, line 13 calculates its prime throughput value, and line 14 sets all variable latency units not on a loop to their maximum values, as suggested by *Finding 4*. Line 15 applies the Tarjan's algorithm to determine regions. Line 16 initiates the traversal of regions, and lines 17-28 involve two iterations over the scenarios within a region. The first iteration, lines 17-21, serves as a preprocessing step. Line 18 sets all variable II unit IIs to prime II, as indicated in Section 4.4. Line 19 calculates the maximum possible throughput for that scenario, and line 20 organizes neighbouring scenarios, where the latency of one variable unit is increased or decreased by 1, into two sets. Lines 22-28 involve the second iteration. Line 23 assesses whether it is necessary to consider optimizing this scenario, using the conclusions found at the end of Section 4.3. Lines 24-26 establish a new set of throughput-related variables and constraints for scenarios requiring optimization and incorporate the previously calculated maximum possible throughput into the constraints. Line 30 concludes the variable and constraint generation process, and line 31 sets the objective to minimize buffer area costs. Lines 32-33 solve the optimization model and save the result of the buffer placement.

To conclude, this section thoroughly discusses the optimization process for variable latency/II units using pseudocode, utilizing all conclusions in previous sections. It is important to note that the process for selecting scenarios to optimize is based on an II-elastic CFDFC, meaning the circuit contains no PL. For circuits with PL, a model that considers all scenarios without filtering can still yield the optimal solution that satisfies the maximum possible throughput while minimizing area costs. However, the runtime required to solve the optimization model may be considerable.

5 Evaluation

This section discusses the effectiveness of our optimization model to maximize throughput and minimize area overhead under fixed unit latency/II. Section 4, which considers variable units, is not addressed here and is left for future research. First, we compare the results of our optimization model with [1]. Then, we analyze the reasons for the differences in area by examining the types and quantities of buffers used in each work. Finally, we will discuss the runtime of our work.

5.1 Methodology

We here describe a complete workflow for optimizing dataflow circuits. After we have a dataflow circuit and CFDFCs identified, we apply the optimization model from Section 3.3 to determine the buffer placement and sizes that satisfy the target clock period and maximize the loop throughputs, we employ the cost function from Equation (20). The weights of each throughput term are proportional to the execution number of the corresponding CFDFC in the program profile. To simplify model complexity, we excluded CFDFCs from our objective if their weight, i.e., w_d in Equation (20), fell below 0.15. We choose the constant value of $\lambda = 3 \times 10^{-5}$ (see Equation (20)) to account for the minimization of buffer areas. We have chosen the *look-up table* (LUT) utilization as the criterion for area cost.

To encourage the optimization model to prefer cheaper buffers, we assign weights to the buffer costs based on the area report from the Vivado synthesis tool [5]. The area weights, denoted as A_i in Equation (20), for the buffer types OB, TB, FT, PL, and DVR (a new type that cuts timing paths for all three signal types) are set to 2, 20, 25, 3, and 2.1, respectively. Notably, multiple slots in a pipeline incur the same LUT cost as a single slot. We present our results for two target periods: 6 ns and 4 ns.

To evaluate our technique, we use ModelSim [6] to measure the throughput and to verify functional correctness. We obtain the timing slacks and resource usages after Vivado synthesis. We use Gurobi Optimizer [7] to solve the model.

5.2 Baseline Descriptions

In this study, we compare our optimization model against two established baseline buffer insertion strategies to evaluate the throughput, timing performance, and area overhead. The baselines selected are as follows:

- Dynamatic [8]: A publicly available implementation, which is publicly available on GitHub. The implementation has some inconsistency compared with the method described in [1]. For the exact version, see the specific commit referenced.
- TRETs: Our re-implementation of [1]. We specifically employed a timing model with three signal paths from our work and substituted the 1-slot non-transparent buffer from [1] with a DVR buffer. These adjustments were made to align the external conditions with our model.

5.3 Comparison

We base our comparison on various kernels obtained from the literature [9, 10]. The benchmarks we consider contain pipelined computational units and exhibit different loop properties and organizations. In the following parts, we will compare the performance and area overhead, and analyze how the usage of different buffer types impacts the area overhead.

LUTs as Memory/Shift Registers are commonly employed in practical applications. In area comparison, we will present results both with and without them. Excluding them allows for a clear distinction between logic and memory, while including them demonstrates the adaptability of our model to this technology. In other parts, the presence or absence of LUTs as Memory/Shift Registers has nothing to do with our conclusions; therefore, we will only report the results where they are excluded.

Performance Comparison

In this part, we compare the circuit performance between our model and two baseline strategies. Tables 1 and 2 demonstrate the circuit performance of Dynamatic, our model, and TRETs across nine benchmarks. The number of cycles required to complete the benchmark significantly characterizes the throughput. Table 1 presents the results for a target clock period of 6 ns, while Table 2 shows the results for a 4 ns period.

In Table 1, for five benchmarks, the cycle numbers for all three strategies are nearly the same, indicating they achieve equal throughput. Minor differences are merely due to the number of cycles required to establish a pipeline steady state. In four benchmarks, our work and TRETs demonstrate a significant reduction in cycle numbers compared to Dynamatic, indicating that these two strategies can explore higher throughput. Overall, our work does not exhibit any throughput disadvantages compared to other strategies. In terms of timing, our work has the largest timing slack in five benchmarks, moderate slack in three, and the least slack in one benchmark. Therefore, our work also shows no disadvantage in timing performance.

Table 2 represents the same kind of results under a 4 ns clock period target. It shows that, in terms of throughput, unlike previously, our work exhibits an obviously higher cycle number in the "video_filter" and "image_resize" benchmarks compared to the other two strategies, indicating that our work did not achieve optimal throughput. Investigations on these two benchmarks indicate that this issue stems from certain units not performing as expected according to the theoretical model, conflicting with the PL's low elasticity. For example, in "video_filter", a "fork" unit receives a valid signal from the upstream circuit every cycle but fails to pass the signal downstream as anticipated in one particular cycle. Consequently, token occupancy in a downstream PL fluctuates. Since the optimization model prescribes very narrow ranges for PLs' token occupancy, this fluctuation exceeds the prescribed range, resulting in reduced throughput. Similarly, in "image_resize", an "mc_store" unit fails to emit a ready signal when theoretically expected, causing a one-cycle blockage in the upstream PL and fluctuating token occupancy, which results in reduced throughput. Such issues are likely to occur at multiple parts within the circuit and merit further detailed investigation.

Benchmark	Cycle Num.			Slack		
	<i>DYN</i>	#	<i>TRETS</i>	<i>DYN</i>	#	<i>TRETS</i>
fir	1014	1014	1014	2.209	1.324	1.451
iir	5007	4009	4009	1.066	2.750	1.731
if_loop_1	213	212	210	0.617	1.372	1.161
if_loop_2	211	210	210	0.619	1.090	1.253
sumi3_mem	1019	1018	1018	1.450	2.268	2.254
vedio_filter	1368	949	948	0.793	0.849	0.757
matvec	10119	10119	10118	1.569	0.512	0.482
image_resize	1065	945	944	1.267	1.997	0.925
matrix	40548	27948	27948	1.103	0.979	0.735

Table 1: Cycle numbers and timing slacks under clock period of 6 ns. *DYN*, #, *TRETS* represent the result from Dynamic, our work and *TRETS* respectively.

Benchmark	Cycle Num.			Slack		
	<i>DYN</i>	#	<i>TRETS</i>	<i>DYN</i>	#	<i>TRETS</i>
fir	2015	2014	2014	0.580	0.799	1.229
iir	5007	4009	4009	0.158	0.365	0.010
if_loop_1	216	216	215	0.604	-0.676	-0.691
if_loop_2	212	213	212	-0.107	0.320	0.617
sumi3_mem	2018	2018	2017	0.429	0.802	1.043
vedio_filter	1847	2327	1940	-0.675	-0.084	0.304
matvec	20118	20218	20117	-0.522	0.003	0.628
image_resize	1843	2249	1843	0.186	-0.004	0.596
matrix	54977	54977	54976	-0.752	-0.268	0.606

Table 2: Cycle numbers and timing slacks under clock period of 4 ns. *DYN*, #, *TRETS* represent the result from Dynamic, our work and *TRETS* respectively.

Benchmark	Utilization						Savings (%)			
	<i>DYN</i>		#		<i>TRETS</i>		<i>TRETS</i>		<i>DYN</i>	
	LUT	Reg.	LUT	Reg.	LUT	Reg.	LUT	Reg.	LUT	Reg.
fir	471	539	434	535	460	544	5.7	1.7	7.9	0.7
iir	666	869	621	892	677	858	8.3	-4.0	6.8	-2.6
if_loop_1	521	630	558	640	625	708	10.7	9.6	-7.1	-1.6
if_loop_2	543	596	570	596	599	634	4.8	6.0	-5.0	0.0
sumi3_mem	465	572	339	547	455	601	25.5	9.0	27.1	4.4
vedio_filter	1593	1550	1243	1563	1585	1648	21.6	5.2	22.0	-0.8
matvec	745	881	533	637	705	747	24.4	14.7	28.5	27.7
image_resize	620	751	551	726	700	755	21.3	3.8	11.1	3.3
matrix	870	863	646	651	756	672	14.6	3.1	25.7	24.6

Table 3: Area utilization and savings under clock period of 6 ns. DYN, #, TRETS represent the result from Dynamatic, our work and TRETS respectively.

Area Comparison

In this part, we compare the area overhead between our model and two baseline strategies. Tables 3-6 presents the area overhead of the three strategies, characterized by the number of LUTs and Registers utilized. Tables 3 and 5 present the results for a target clock period of 6 ns, while Tables 4 and 6 show the results for a 4 ns period. Tables 3 and 4 present the results excluding LUTs as Memory/Shift Registers, while Tables 5 and 6 include them.

Generally, LUTs construct the logic within the circuit, while Registers store variable values and states. Given that LUTs are larger and fewer in quantity, we focus primarily on their usage. The four rightmost columns of each table show the amount of LUTs and Registers saved by our work relative to the other two strategies. Table 3 shows that, compared to TRETS, our work saved between 4.8% and 25.5% of LUT usage across nine benchmarks, with a geometrical mean of savings at 12.94%, and register usage was reduced in all but one benchmark. Compared to Dynamatic, our work showed less optimal area usage in two similar benchmarks, which are "if_loop_1" and "if_loop_2", but maintained good performance in the rest, with LUT savings ranging from 6.8% to 28.5%, with a geometrical mean of savings at 15.93%.

When operating under a target clock period of 4 ns, Table 4 indicates that our work incurs larger area overhead in three benchmarks compared to TRETS, marking a slight decline from the results shown in Table 3. For benchmarks where our work reduces area overhead, the geometrical mean of LUT savings is 8.25%. In comparison with Dynamatic, the geometrical mean is 13.16%. These results suggest that our approach performs less effectively under a shorter clock period.

Tables 5 and 6 show the area overhead comparisons under target clock period of 6ns and 4 ns respectively when LUTs as Memory/Shift Registers are utilized. It can be observed that our work still maintains considerable area savings in most benchmarks, thereby demonstrating that it is not adversely affected by this technique. The geometrical mean of LUT savings is 7.44%

Benchmark	Utilization						Savings (%)			
	<i>DYN</i>		#		<i>TRETS</i>		<i>TRETS</i>		<i>DYN</i>	
	LUT	Reg.	LUT	Reg.	LUT	Reg.	LUT	Reg.	LUT	Reg.
fir	474	563	450	533	456	536	1.3	0.6	5.06	5.33
iir	667	901	733	891	680	859	-7.8	-3.7	-9.90	1.11
if_loop_1	623	734	577	743	563	750	-2.5	0.9	7.38	-1.23
if_loop_2	501	660	558	605	547	608	-2.0	0.5	-11.38	8.33
sumi3_mem	445	487	345	512	429	525	19.6	2.5	22.47	-5.13
vedio_filter	1510	1430	1202	1507	1418	1372	15.2	-9.8	20.40	-5.38
matvec	722	743	560	653	605	627	7.4	-4.1	22.44	12.11
image_resize	634	627	512	740	573	591	10.6	-25.2	9.62	5.74
matrix	811	725	661	644	738	644	10.4	0.0	18.50	11.17

Table 4: Area utilization and savings under clock period of 4 ns. DYN, #, TRETS represent the result from Dynamatic, our work and TRETS respectively.

compared to TRETS and 8.61% compared to Dynamatic under a 6 ns clock period. Under a 4 ns clock period, the means are 8.15% and 15.75% respectively.

In the next part, we will analyze the reasons behind these results to elucidate why our approach achieves lower area overhead in most benchmarks and fails to produce satisfactory outcomes in others.

Benchmark	Utilization						Savings (%)			
	<i>DYN</i>		#		<i>TRETS</i>		<i>TRETS</i>		<i>DYN</i>	
	LUT	Reg.	LUT	Reg.	LUT	Reg.	LUT	Reg.	LUT	Reg.
fir	470	531	436	528	449	535	2.9	1.3	7.23	0.56
iir	666	869	625	863	631	860	1.0	-0.3	6.16	0.69
if_loop_1	521	630	560	636	626	704	10.5	9.7	-7.49	-0.95
if_loop_2	543	596	570	596	599	634	4.8	6.0	-4.97	0.00
sumi3_mem	437	426	373	469	427	457	12.6	-2.6	14.65	-10.09
vedio_filter	1547	1187	1322	1318	1557	1371	15.1	3.9	14.54	-11.04
matvec	699	579	557	577	641	587	13.1	1.7	20.31	0.35
image_resize	612	531	607	565	703	605	13.7	6.6	0.82	-6.40
matrix	841	587	655	606	762	628	14.0	3.5	22.12	-3.24

Table 5: Area utilization and savings under clock period of 6 ns. DYN, #, TRETS represent the result from Dynamatic, our work and TRETS respectively. LUTs as Memory/Shift Registers are employed.

Benchmark	Utilization						Savings (%)			
	<i>DYN</i>		#		<i>TRETS</i>		<i>TRETS</i>		<i>DYN</i>	
	LUT	Reg.	LUT	Reg.	LUT	Reg.	LUT	Reg.	LUT	Reg.
fir	482	559	445	529	460	532	3.3	0.6	7.68	5.37
iir	667	901	692	847	737	859	6.1	1.4	-3.75	5.99
if_loop_1	629	732	581	735	578	746	-0.5	1.5	7.63	-0.41
if_loop_2	501	660	554	633	547	608	-1.3	-4.1	-10.58	4.09
sumi3_mem	461	415	347	500	435	454	20.2	-10.1	24.73	-20.48
vedio_filter	1550	1233	1205	1312	1433	1320	15.9	0.6	22.26	-6.41
matvec	728	580	565	589	603	595	6.3	1.0	22.39	-1.55
image_resize	692	539	554	563	550	565	-0.7	0.4	20.52	-4.82
matrix	806	585	675	608	727	626	7.2	2.9	16.25	-3.93

Table 6: Area utilization and savings under clock period of 4 ns. DYN, #, TRETS represent the result from Dynamatic, our work and TRETS respectively. LUTs as Memory/Shift Registers are employed.

Analysis of Buffer Usage

In this part, we analyze the reasons for the savings in area usage by our work and the suboptimal performance in some benchmarks.

Tables 7 and 8 detail the total number of buffer slots placed and the total bitwidth of data stored by buffers for Dynamatic, our work, and TRETTS. Table 7 aligns with the results from Table 3, while Table 8 corresponds to the results in Table 4. The number of slots is associated with LUT usage, and the bitwidth is linked to register usage. Both tables indicate that despite a similar total number of slots used by the three strategies, our approach prioritizes the use of PL whenever feasible; since PLs typically have lower LUT usage compared to other types, this strategy systematically leads to LUT savings.

In some benchmarks, our solution incurs a higher area overhead despite using PL instead of more costly types. This outcome can be attributed to unexploited logic optimization opportunities across different dataflow units, which our model currently overlooks. Incorporating this aspect into our model could provide more accurate predictions [11].

Studies have shown that even slight modifications, such as changing the sequence of buffers within the same channel—a detail our model does not consider—can significantly influence area costs, thus highlighting the limitations of our current modeling approach.

Benchmark	OB		TB		FT/PL		DVR		Total	
	Slots	Bits	Slots	Bits	Slots	Bits	Slots	Bits	Slots	Bits
<i>DYN</i>	OB		TB		FT		DVR			
fir	3	45	3	45	8	8	0	0	14	98
iir	5	109	5	109	0	0	0	0	10	218
if_loop_1	8	194	4	35	0	0	0	0	12	229
if_loop_2	8	163	2	33	0	0	0	0	10	196
sumi3_mem	5	46	3	44	22	146	0	0	30	236
vedio_filter	9	88	6	84	63	486	0	0	78	658
matvec	7	92	3	18	42	347	0	0	52	457
image_resize	7	86	5	78	15	290	0	0	27	454
matrix	11	80	6	27	37	349	0	0	54	456
#	OB		TB		PL		DVR			
fir	3	44	3	44	9	9	0	0	15	97
iir	6	79	0	0	5	160	4	77	15	316
if_loop_1	4	128	2	64	6	6	2	33	14	231
if_loop_2	5	98	2	64	0	0	2	33	9	195
sumi3_mem	3	45	3	45	20	144	0	0	26	234
vedio_filter	9	149	5	77	59	549	0	0	73	775
matvec	8	92	4	49	26	178	0	0	38	319
image_resize	8	98	4	45	13	236	0	0	25	379
matrix	13	87	5	51	26	94	0	0	44	232
<i>TRETS</i>	OB		TB		FT		DVR			
fir	5	46	5	46	7	7	0	0	17	99
iir	2	33	2	33	0	0	8	153	12	219
if_loop_1	7	162	7	162	2	2	2	33	18	359
if_loop_2	5	98	5	98	0	0	2	33	12	229
sumi3_mem	6	79	6	79	17	110	0	0	29	268
vedio_filter	19	198	19	198	36	310	4	72	78	778
matvec	9	100	9	100	18	94	4	50	40	344
image_resize	9	141	9	141	7	148	3	40	28	470
matrix	10	74	10	74	19	58	7	28	46	234

Table 7: Buffer slot and bit count under clock period of 6 ns for Dynamatic, our work, and TRETS. The third buffer type in this table can be either FT or PL. Dynamatic and TRETS only used FT while our work only used PL.

Benchmark	OB		TB		FT/PL		DVR		Total	
	Slots	Bits	Slots	Bits	Slots	Bits	Slots	Bits	Slots	Bits
<i>DYN</i>	OB		TB		FT		DVR			
fir	5	88	2	33	4	4	0	0	11	125
iir	7	153	4	97	0	0	0	0	11	250
if_loop_1	11	290	3	34	2	2	0	0	16	326
if_loop_2	9	257	2	2	0	0	0	0	11	259
sumi3_mem	6	89	1	1	10	72	0	0	17	162
vedio_filter	15	170	9	76	29	396	0	0	53	642
matvec	12	117	2	9	21	253	0	0	35	379
image_resize	12	105	6	73	6	116	0	0	24	294
matrix	15	89	8	35	17	140	0	0	40	264
<i>#</i>	OB		TB		PL		DVR			
fir	3	45	0	0	6	6	3	44	12	95
iir	6	79	0	0	5	160	4	76	15	315
if_loop_1	7	224	0	0	8	8	4	97	19	329
if_loop_2	5	98	0	0	3	3	4	97	12	198
sumi3_mem	3	45	0	0	17	110	3	44	23	199
vedio_filter	14	215	0	0	56	418	5	77	75	710
matvec	10	133	0	0	26	82	4	49	40	264
image_resize	9	93	0	0	15	290	4	45	28	428
matrix	16	128	0	0	20	92	5	51	41	271
<i>TRETS</i>	OB		TB		FT		DVR			
fir	4	35	4	35	2	2	2	23	12	95
iir	2	33	2	33	0	0	8	153	12	219
if_loop_1	2	2	2	2	2	2	11	321	17	327
if_loop_2	1	1	1	1	0	0	9	195	11	197
sumi3_mem	5	67	5	67	5	36	3	25	18	195
vedio_filter	16	191	16	191	4	52	10	105	46	539
matvec	6	51	6	51	4	4	11	110	27	216
image_resize	4	90	4	90	0	0	13	81	21	261
matrix	8	73	8	73	5	17	14	59	35	222

Table 8: Buffer slot and bit count under clock period of 4 ns for Dynamatic, our work, and TRETS. The third buffer type in this table can be either FT or PL. Dynamatic and TRETS only used FT while our work only used PL.

Benchmark	Runtime (s)			
	6 ns clock period		4 ns clock period	
	#	TRETS	#	TRETS
fir	4	<1	1	<1
iir	1	<1	1	<1
if_loop_1	5	<1	2	<1
if_loop_2	5	<1	1	<1
sumi3_mem	2	<1	<1	<1
vedio_filter	3000	<1	3000	21
matvec	964	<1	3000	5
image_resize	18	<1	563	4
matrix	3000	2	3000	43

Table 9: Gurobi Optimizer runtime under a target clock period of 6 ns and 4 ns. # and TRETS represent the runtime of our work and TRETS respectively.

5.4 Runtime

This section discusses the runtime required to solve the optimization model. Table 9 presents the runtime of the Gurobi Optimizer for solving the models from our work and TRETS. For smaller benchmarks, the runtime is within 10 seconds. It is evident that our model consistently requires more runtime because it is not linear but includes quadratic and bilinear components. Therefore, for larger benchmarks, we have set a runtime limit of 3000 seconds. Additionally, the shorter target clock period generally necessitates longer runtime.

In summary, the experimental results demonstrate that our model substantially reduces area overhead while maintaining throughput and timing performance in most cases, thereby validating the theoretical feasibility. However, the deviations present in the model have had adverse effects in certain instances, indicating requirements for improvement.

6 Conclusion

In this work, we have developed a framework for modeling the performance of various types of buffers and incorporating them into the CFDFC throughput constraints. We proposed an optimization model aimed at maximizing throughput while minimizing buffer area costs. Furthermore, we introduced an optimization flow that consistently meets maximum throughput under multiple latency/II scenarios and minimizes buffer area costs. To reduce the operational costs of this optimization flow, we analyzed the impact of variable latency/II units on this optimization model and systematically eliminated scenarios that are unnecessary to consider.

Bibliography

- [1] L. Josipović, S. Sheikhha, A. Guerrieri, P. Jenne, and J. Cortadella, “Buffer placement and sizing for high-performance dataflow circuits,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 15, no. 1, nov 2021. [Online]. Available: <https://doi.org/10.1145/3477053>
- [2] L. Benini, E. Macii, M. Poncino, and G. De Micheli, “Telescopic units: a new paradigm for performance optimization of vlsi designs,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 17, no. 3, pp. 220–232, 1998.
- [3] R. Tarjan and U. Vishkin, “Finding biconnected components and computing tree functions in logarithmic parallel time,” in *25th Annual Symposium on Foundations of Computer Science, 1984.*, 1984, pp. 12–20.
- [4] A. Dasdan and R. Gupta, “Faster maximum and minimum mean cycle algorithms for system-performance analysis,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 17, no. 10, pp. 889–899, 1998.
- [5] *Vivado Design Suite*, Xilinx Inc., 2020. [Online]. Available: <https://docs.xilinx.com/v/u/2019.2-English/ug901-vivado-synthesis>
- [6] Mentor Graphics, “ModelSim,” 2016. [Online]. Available: <https://www.mentor.com/products/fv/modelsim/>
- [7] Gurobi Optimization, LLC, “Gurobi Optimizer Reference Manual,” 2022. [Online]. Available: <https://www.gurobi.com>
- [8] L. Ramirez, “Dynamatic,” accessed on 2024-05-13. Specific commit ID: 905b770. [Online]. Available: <https://github.com/EPFL-LAP/dynamatic>
- [9] R. Kastner, J. Matai, and S. Neuendorffer, “Parallel programming for fpgas,” 2018. [Online]. Available: <https://arxiv.org/abs/1805.03648>
- [10] W. H. Press, *Numerical recipes 3rd edition: The art of scientific computing*. Cambridge university press, 2007.
- [11] H. Wang, C. Rizzi, and L. Josipović, “Mapbuf: Simultaneous technology mapping and buffer insertion for hls performance optimization,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.